DS	**Session**	Sorting	**Question Information**	Level 1 Challenge 13

Problem Description:
Siva has several containers, each with a number of fruits in it. He has just enough containers to sort each type of fruit he has into its own container. Siva wants to sort the fruits using his sort method.

Siva wants to perform some number of swap operations such that:

Each container contains only fruits of the same type.
No two fruits of the same type are located in different containers.

Function Description

organizingContainers has the following parameter(s):

int containter[n][m]: a two dimensional array of integers that represent the number of balls of each color in each container

Constraints:
$1 \leq q \leq 10$
$1 \leq n \leq 100$
$0 \leq containers[i][j] \leq 10^9$

Input Format:
The first line contains an integer 'q', the number of queries.

Each of the next 'q' sets of lines is as follows:

The first line contains an integer 'n', the number of containers (rows) and ball types (columns).
Each of the next 'n' lines contains 'n' space-separated integers describing row containers[i].

Output Format:
For each query, print Possible on a new line if David can satisfy the conditions above for the given matrix. Otherwise, print Impossible.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
2
1 1
1 1
2
0 2
1 1
```

EXPECTED OUTPUT

```
Possible
Impossible
```

Test Case 2

INPUT (STDIN)

```
2
2
2 1
1 2
2
1 2
2 1
```

EXPECTED OUTPUT

```
Possible
Possible
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void insertionSort(long int
*p,long int n)
```

Test Case 2

KEYWORD

```
for(i=0;i<n;i++)
```

Test Case 3

KEYWORD

```
while(q--)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
5
```

Test Case 2

TOKEN COUNT

```
450
```

Test Case 3

NLOC

```
63
```

Code

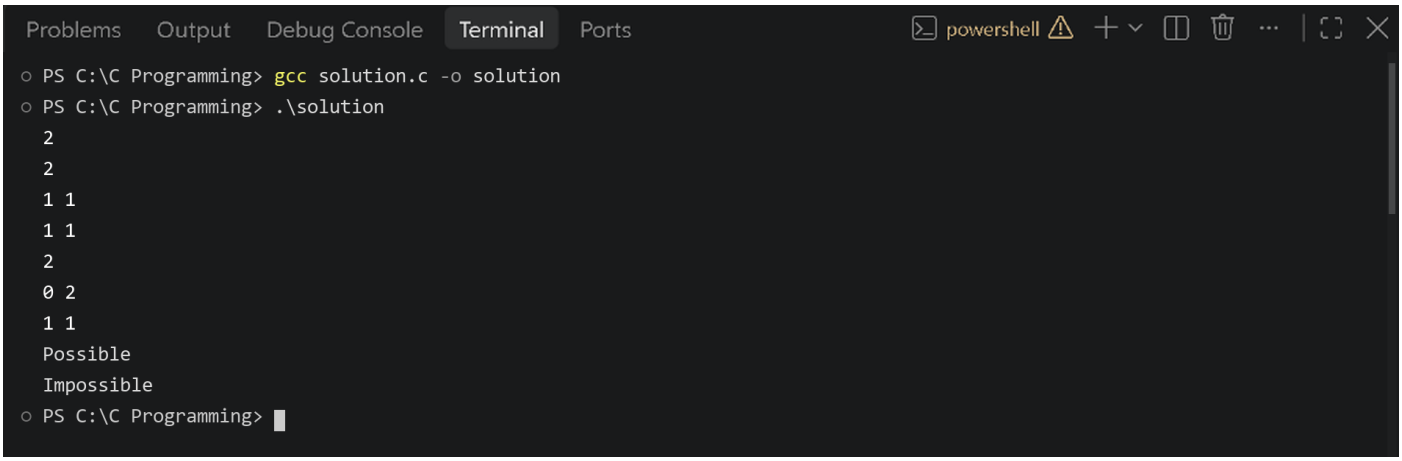
solution.c

```c
#include <stdio.h>

void insertionSort(long int *p, long int n) {
    int i, j;
    long int key;
    for (i = 1; i < n; i++) {
        key = p[i];
        j = i - 1;
        while (j >= 0 && p[j] > key) {
            p[j + 1] = p[j];
            j = j - 1;
        }
        p[j + 1] = key;
    }
}

int main() {
    int q;
    if (scanf("%d", &q) != 1) return 0;
    while (q--) {
        long int n;
        scanf("%ld", &n);
        long int rowSum[n], colSum[n];
        for (int i = 0; i < n; i++) { rowSum[i] = 0; colSum[i] = 0; }
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                long int val;
                scanf("%ld", &val);
                rowSum[i] += val;
                colSum[j] += val;
            }
        }
        insertionSort(rowSum, n);
        insertionSort(colSum, n);
        int possible = 1;
        for (int i = 0; i < n; i++) {
            if (rowSum[i] != colSum[i]) { possible = 0; break; }
        }
        if (possible) printf("Possible\n");
        else printf("Impossible\n");
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
2
2
1 1
1 1
2
0 2
1 1
Possible
Impossible
PS C:\C Programming>